

Improving Deep Neural Networks

DL Specialization, Course 2 — detailed notes (Weeks 1–3)

Regularization · optimization · hyperparameter tuning · batch norm · TensorFlow

1. Week 1a — Setting up the ML application

1.1 Train / Dev / Test sets

You can't get all hyperparameters right first try — it's an iterative loop: *idea* → *code* → *experiment*. Split the data three ways:

- **Training** set — fit the parameters (largest).
- **Dev** (cross-validation) set — tune hyperparameters, pick models.
- **Test** set — final unbiased estimate of performance.

Split ratios: classic ML used 60/20/20 (or 70/30). But with 10^6+ examples, dev/test need only be big enough to *evaluate*, so 98/1/1 or even 99.5/0.25/0.25.

Same distribution: dev and test must come from the same distribution (e.g. don't train on web images and test on phone images). It's OK to have only a dev set (no test) — but then call it a dev set, not a test set.

1.2 Bias / Variance

Diagnose by comparing train error and dev error (assuming human/Bayes error ≈ 0):

Train err	Dev err	Diagnosis
1%	11%	high variance (overfit)
15%	14%	high bias (underfit)
15%	30%	high bias <i>and</i> high variance
0.5%	1%	low bias, low variance (good)

If the optimal (Bayes) error isn't near 0, compare against that baseline instead.

1.3 Basic recipe for machine learning

1. **High bias?** (train error high) → bigger network, train longer, better architecture/optimizer.
2. **High variance?** (dev error $\gg$ train) → more data, regularization, better architecture.
3. Repeat until both are low.

In the deep-learning era the old “bias–variance tradeoff” is weaker: a bigger network almost always cuts bias without hurting variance (given regularization), and more data cuts variance without hurting bias. *Training a bigger network rarely hurts* (except compute).

2. Week 1b — Regularization

2.1 L2 regularization

Add a penalty on large weights to reduce variance. For a network:

$$J = \frac{1}{m} \sum_i L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_l \|W^{[l]}\|_F^2, \quad \|W^{[l]}\|_F^2 = \sum_{i,j} (W_{ij}^{[l]})^2.$$

This $\|\cdot\|_F$ is the **Frobenius norm** (sum of squares of all entries). L2 is used far more than L1 (which drives weights to exact zeros, sparsifying).

Weight decay

Backprop gains an extra term, so the update shrinks W every step:

$$dW^{[l]} = (\text{backprop term}) + \frac{\lambda}{m} W^{[l]}, \quad W^{[l]} := \left(1 - \frac{\alpha\lambda}{m}\right) W^{[l]} - \alpha(\text{backprop term}).$$

The factor $(1 - \alpha\lambda/m) < 1$ is why L2 is called “weight decay.”

Why it reduces overfitting

Two intuitions: (1) large $\lambda \Rightarrow$ many weights $\rightarrow 0 \Rightarrow$ a simpler, closer-to-linear network; (2) with tanh, small weights keep z small, so activations stay in the roughly-linear region of tanh $\Rightarrow$ a simpler decision boundary. Tune λ on the dev set.

2.2 Dropout

Randomly “drop” (zero out) units each iteration with probability $1 - \text{keep_prob}$. **Inverted dropout:**

```
d3 = np.random.rand(*a3.shape) < keep_prob    # boolean mask
a3 = np.multiply(a3, d3)                        # drop units
a3 = a3 / keep_prob                             # scale up to keep expected value the same
```

- **Only at training time.** At test time use no dropout (else predictions get noisy).
- Apply the same mask in forward *and* backward pass of that iteration.
- Dividing by `keep_prob` keeps activations’ expected value unchanged (so test-time scaling is automatic).

Why it works: a unit can’t rely on any single input (any might vanish), so it spreads weights out — similar effect to L2. You can set a lower `keep_prob` on layers more prone to overfitting (bigger layers). Common in computer vision (data-hungry).

Downside: J is no longer well-defined each step, so you can’t plot a clean cost curve. To debug, set all `keep_prob=1`, confirm J decreases monotonically, then re-enable dropout.

2.3 Other regularization methods

- **Data augmentation:** cheap extra data — flip/rotate/crop images, distort digits. Not as good as fresh data but regularizes.
- **Early stopping:** plot train and dev cost; stop where dev cost bottoms out. Downside: it couples optimizing J with not-overfitting (violates “orthogonalization”). Andrew prefers L2, though early stopping avoids searching λ .

3. Week 1c — Setting up the optimization problem

3.1 Normalizing inputs

Speeds up training a lot. Two steps (use *train*-set statistics for all sets):

$$\mu = \frac{1}{m} \sum_i x^{(i)}, \quad x := x - \mu; \quad \sigma^2 = \frac{1}{m} \sum_i x^{(i)2}, \quad x := x/\sigma.$$

Un-normalized features make the cost contours long and thin, so gradient descent zig-zags with a small α . Normalized $\Rightarrow$ symmetric bowl $\Rightarrow$ larger α , faster convergence.

3.2 Vanishing / exploding gradients

In a deep net with (roughly) linear activations and $b = 0$, $\hat{y} \approx W^{[L]}W^{[L-1]} \dots W^{[1]}x$. If each $W \approx 1.5I$ the output scales like 1.5^L (explodes); if $\approx 0.5I$, like 0.5^L (vanishes). So activations and gradients grow/shrink *exponentially* in depth — training stalls (tiny steps) or blows up (NaN). ResNet trains 152 layers precisely because it fights this.

3.3 Weight initialization for deep networks

Partial fix: scale the random init so variance stays ≈ 1 . For a neuron with n inputs, want $\text{Var}(W) \approx 1/n^{[l-1]}$:

$$\begin{aligned} \text{tanh (Xavier): } W^{[l]} &= \text{randn}(\cdot) \times \sqrt{1/n^{[l-1]}}, \\ \text{ReLU (He): } W^{[l]} &= \text{randn}(\cdot) \times \sqrt{2/n^{[l-1]}}. \end{aligned}$$

He initialization ($\sqrt{2/n^{[l-1]}}$) works best with ReLU — a strong partial cure for vanishing/exploding gradients.

3.4 Gradient checking

Verify your backprop is correct using a numerical gradient:

$$\frac{\partial J}{\partial \theta_i} \approx \frac{J(\dots, \theta_i + \varepsilon, \dots) - J(\dots, \theta_i - \varepsilon, \dots)}{2\varepsilon}, \quad \varepsilon \approx 10^{-7}.$$

Reshape all W, b into one vector θ ; compute $d\theta_{\text{approx}}$ and compare to backprop's $d\theta$:

$$\text{ratio} = \frac{\|d\theta_{\text{approx}} - d\theta\|_2}{\|d\theta_{\text{approx}}\|_2 + \|d\theta\|_2}.$$

$< 10^{-7}$ great; $\approx 10^{-5}$ inspect; $\geq 10^{-3}$ likely a bug.

Grad-check notes: use only for *debugging* (slow), not training. Include the $\frac{\lambda}{m}W$ term if regularizing. Doesn't work with dropout (turn it off first). Run after a few iterations, not just at init.

4. Week 2 — Optimization Algorithms

4.1 Mini-batch gradient descent

With huge m (say 5M), one gradient step over all data is very slow. Split into mini-batches (e.g. 1000 each): $X^{\{t\}}, Y^{\{t\}}$.

```
for t in range(num_batches):           # one pass = one "epoch"
    forward_prop(X{t}); compute cost on X{t}
    backward_prop(); update_parameters()
```

Sizes: batch = m is batch GD (slow/epoch); batch = 1 is stochastic GD (noisy, loses vectorization); *between* is mini-batch — best of both. Guidelines: small set (< 2000) use batch GD; else a power of 2 (64, 128, 256, 512) that fits in memory. Mini-batch cost is noisy but trends down.

4.2 Exponentially weighted averages

Building block for the better optimizers. Smooths a noisy series:

$$V_t = \beta V_{t-1} + (1 - \beta)\theta_t, \quad \text{averages over } \approx \frac{1}{1 - \beta} \text{ terms.}$$

$\beta=0.9 \rightarrow \text{last } \sim 10$; $\beta=0.98 \rightarrow \sim 50$. **Bias correction** (early values start too low since $V_0=0$): divide by $(1 - \beta^t)$, which $\rightarrow 1$ as t grows.

4.3 Momentum

Average the gradients (EWA) and step with the average — damps oscillation, speeds progress:

$$\begin{aligned} V_{dW} &= \beta V_{dW} + (1 - \beta) dW, & W &:= W - \alpha V_{dW}, \\ V_{db} &= \beta V_{db} + (1 - \beta) db, & b &:= b - \alpha V_{db}. \end{aligned}$$

$\beta = 0.9$ works well; bias correction usually skipped in practice.

4.4 RMSprop

Divide each update by the root-mean-square of recent gradients — slows the fast (oscillating) direction, speeds the slow one:

$$S_{dW} = \beta S_{dW} + (1 - \beta) dW^2, \quad W := W - \alpha \frac{dW}{\sqrt{S_{dW} + \varepsilon}},$$

(squaring is element-wise; $\varepsilon \approx 10^{-8}$ avoids divide-by-zero). Lets you use a larger α .

4.5 Adam = Momentum + RMSprop

The workhorse optimizer. Keeps both averages, with bias correction:

$$\begin{aligned} V_{dW} &= \beta_1 V_{dW} + (1 - \beta_1) dW, & S_{dW} &= \beta_2 S_{dW} + (1 - \beta_2) dW^2, \\ V^{\text{corr}} &= V / (1 - \beta_1^t), & S^{\text{corr}} &= S / (1 - \beta_2^t). \\ W &:= W - \alpha \frac{V_{dW}^{\text{corr}}}{\sqrt{S_{dW}^{\text{corr}} + \varepsilon}}. \end{aligned}$$

Defaults: $\beta_1=0.9$, $\beta_2=0.999$, $\varepsilon=10^{-8}$; tune only α . (Adam = “adaptive moment estimation.”)

4.6 Learning rate decay

Shrink α over time so steps get smaller near the optimum (mini-batch GD only oscillates around it otherwise). E.g. $\alpha = \frac{1}{1 + \text{decay} \cdot \text{epoch}} \alpha_0$, or $0.95^{\text{epoch}} \alpha_0$, or $\frac{k}{\sqrt{\text{epoch}}} \alpha_0$. Lower priority for Andrew than picking a good α .

4.7 The problem of local optima

In high dimensions, true bad local optima are rare — a point must be a minimum in *every* direction, which is unlikely. You’re far more likely to hit **saddle points** (fine) and **plateaus** (flat regions where learning crawls). Momentum/RMSprop/Adam help escape plateaus.

5. Week 3a — Hyperparameter Tuning

5.1 Tuning process

Rough priority order (Andrew's):

1. Learning rate α (most important).
2. Momentum β , # hidden units, mini-batch size.
3. # layers, learning-rate decay.
4. Adam's $\beta_1, \beta_2, \varepsilon$ (rarely tuned).

Use random search, not a grid — you cover more distinct values of the important hyperparameters.
Coarse-to-fine: zoom into the promising region and sample more densely.

5.2 Appropriate scale

Don't sample uniformly on a linear scale when the range spans orders of magnitude — use a **log scale**.
For $\alpha \in [10^{-4}, 1]$:

```
r = -4 * np.random.rand()    # r in [-4, 0]
alpha = 10**r
```

For $\beta \in [0.9, 0.999]$, sample $1 - \beta \in [0.001, 0.1]$ on a log scale (small changes near 0.999 matter a lot).

5.3 Pandas vs. Caviar

- **Panda:** few resources — babysit one model, nudge hyperparameters daily.
- **Caviar:** many resources — train many models in parallel, pick the best.

6. Week 3b — Batch Normalization

6.1 Normalizing activations

Extend input-normalization to *every hidden layer*: normalize $Z^{[l]}$ so the next layer trains faster. Per mini-batch, for layer l :

$$\begin{aligned}\mu &= \frac{1}{m} \sum_i z^{(i)}, & \sigma^2 &= \frac{1}{m} \sum_i (z^{(i)} - \mu)^2, \\ z_{\text{norm}}^{(i)} &= \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \varepsilon}}, & \tilde{z}^{(i)} &= \gamma z_{\text{norm}}^{(i)} + \beta.\end{aligned}$$

γ, β are *learnable* (updated like weights) — they let the layer choose its own mean/variance instead of being forced to 0/1. (If $\gamma = \sqrt{\sigma^2 + \varepsilon}$ and $\beta = \mu$, you recover z .) Normalizing z (pre-activation) is the common choice.

6.2 Fitting into a network

$\beta^{[l]}, \gamma^{[l]}$ join $W^{[l]}$ as trainable params (updated by GD/Adam).

Batch norm subtracts the mean, so the bias $b^{[l]}$ is cancelled out — **drop** $b^{[l]}$ (or set it to 0); $\beta^{[l]}$ plays its role. Params per layer become $W^{[l]}, \gamma^{[l]}, \beta^{[l]}$.

6.3 Why it works

Two reasons: (1) same as normalizing inputs — keeps each layer’s inputs on a stable scale; (2) reduces **covariate shift** — as earlier layers change, later layers see a more stable input distribution, so they can learn more independently. Minor bonus: per-mini-batch mean/variance add noise (like dropout), giving a slight *regularization* effect — but don’t rely on it for regularization.

6.4 Batch norm at test time

At test you may process one example, so per-batch μ, σ^2 don’t make sense. Instead track a **running (exponentially weighted) average** of μ, σ^2 across training mini-batches and use those at test time. Frameworks do this automatically.

7. Week 3c — Multi-class & Softmax

7.1 Softmax regression

Generalizes logistic regression to C classes. Output layer has C units; given $z^{[L]}$:

$$t = e^{z^{[L]}}, \quad a^{[L]} = \frac{t}{\sum_{k=1}^C t_k} \Rightarrow a_k = P(y=k \mid x), \quad \sum_k a_k = 1.$$

“Softmax” softens the values (vs. “hard max” which outputs 1 for the top class, 0 elsewhere). $C = 2$ reduces to logistic regression.

7.2 Training a softmax classifier

Loss and cost:

$$L(y, \hat{y}) = - \sum_{j=1}^C y_j \log \hat{y}_j, \quad J = \frac{1}{m} \sum_i L(y^{(i)}, \hat{y}^{(i)}).$$

Since y is one-hot, $L = -\log \hat{y}_{(\text{true class})}$ — push up the probability of the correct class. Backprop start: $dZ^{[L]} = \hat{Y} - Y$ (same clean form as before).

8. Week 3d — Programming Frameworks (TensorFlow)

8.1 Why frameworks

Implementing everything in NumPy teaches mechanics, but real work uses frameworks (TensorFlow, PyTorch, Keras, ...). Choose by ease of dev/deployment, speed, and genuine openness. Frameworks also auto-optimize your code.

8.2 TensorFlow essentials

You specify only *forward* prop (the cost); TF computes backprop automatically via its computation graph / autodiff.

```
import tensorflow as tf
w = tf.Variable(0, dtype=tf.float32)
optimizer = tf.keras.optimizers.Adam(0.1)

def cost():
    return w**2 - 10*w + 25          # (w-5)^2, minimized at w=5

for i in range(1000):
    optimizer.minimize(cost, [w])
```

```
print(w.numpy()) # -> ~5.0
```

- **GradientTape** records operations for autodiff; `tape.gradient(cost, vars)` gives the gradients.
- Initialize weights with `tf.keras.initializers` (e.g. Xavier/He); one line swaps the optimizer.
- Frameworks fold softmax into the loss for numerical stability (`from_logits=True`).

Big picture of Course 2: Course 1 got a network *working*; Course 2 makes it work *well*. Diagnose with bias/variance → fix bias with bigger nets, variance with regularization (L2, dropout, data aug) → train faster with normalized inputs, good init (He), mini-batches, and Adam → stabilize deep nets with batch norm → tune hyperparameters (random + log scale, α first) → and let a framework handle backprop.